WEEK NOTES · WEEK 5

# BUILD WITH AI

*Vibecoding & agentic app development.*

You automated workflows. This week you ship **products** — real, functional apps built with Claude Code, AI editors and agentic scaffolding. No CS degree. No waiting for a developer.

## THE IDENTITY SHIFT

AI Systems Operator → AI Product Builder → **You ship real products**

**How to read these notes**

Two sessions, each broken into short topics. Each topic is one idea — what it is, how it works, and why it matters for your build. Read once before class; skim again before you scaffold your capstone.

# THIS WEEK'S SHIFT

From automating workflows to shipping products people can use.

**Weekly objective**

Ship a real, functional app — built with Claude Code, AI editors and agentic scaffolding — and scope a capstone you can finish in Week 6.

## CLAUDE FEATURES APPLIED THIS WEEK

**Claude Code** — the agentic terminal / AI IDE; reads CLAUDE.md and builds.

**MCPs** — tool integrations (GitHub, Supabase, APIs) inside the build.

**Claude API** — Claude as the reasoning brain inside your app.

**Projects & Skills** — full app context across sessions; reusable code patterns.

**Connectors** — live data sources (a user's Notion or Drive) inside the app.

**Plugins** — Claude extensions for development.

## THE TWO SESSIONS

SESSION 1

**How to build with AI — vibecoding, Claude Code, Cursor & Lovable. The foundation, the engine, the tools.**

SESSION 2

**Agentic apps — Claude as the brain, Supabase, deploy & capstone planning.**

# SESSION 1 – THE FOUNDATION

How building with AI actually works, and the mindset behind it.

### How AI code generation works

Code models are trained on code, not just text — they track syntax structure and execution context, so their output can be **run and checked** in a way prose never can. A wrong line fails loudly, which is exactly why you can iterate so fast: the computer tells you when it's wrong.

### The vibecoding philosophy

The loop is **describe intent** → **generate** → **test** → **iterate** → **own**. You are the architect; Claude is the builder. You don't type every line — you decide what "good" looks like, read what Claude produced, test it, and keep it only once it works.

### The build loop

Every build runs the same path: **requirements** → **architecture** → **generate** → **test** → **debug** → **refine** → **deploy**. Naming the stage you're in stops a session from sprawling — you always know the next move, and you don't jump to code before the shape is clear.

### Reading & owning the code

AI-generated code is a strong first draft, not a black box. Read it, find where it might be wrong, and learn enough to extend it. The test of ownership is simple: if you can't explain what a file does, ask Claude to walk you through it until you can.

# SESSION 1 – THE ENGINE

Claude Code and the one file that keeps it on track.

**Claude Code**

An agentic coding environment in your terminal or IDE. It reads your project, edits files, runs commands and opens pull requests — showing you each **diff before it writes**. Install it, open your project folder, and it works inside your real repository, not a sandbox.

**CLAUDE.md — the context file**

A plain-text file at your repo root that Claude Code reads **every session**: the goal, the stack, folder conventions and a short "never do" list. It is the single most important file in any project — a vague CLAUDE.md makes every build drift. Keep it under a page and keep it current.

**Claude Code with MCPs**

MCPs wire the coding environment to **GitHub, Supabase, Notion and external APIs** during development, so Claude can read your repo, read and write your database, and open PRs. You authorise each with a scoped token (never your password) and grant the least access it needs.

**Remember**

CLAUDE.md is your leash and your brief in one. Spend five minutes on it before any build — it pays back the whole session.

# SESSION 1 – THE TOOLS

The editors and scaffolders that turn intent into a running app.

### Cursor

An AI-first editor with four ways to build: **tab-complete** for the next line, **Cmd-K** for an inline edit, a **chat sidebar** for questions, and **composer** for multi-file changes. Reach for each at the matching scale of change.

### Lovable

Full-stack app generation from a prompt: describe the app and it scaffolds the **UI and a database together**, then you iterate component by component. Good when you want a working app fast and will refine from there.

### Bolt.new

Instant web-app scaffolding from a single prompt — the fastest way to get something on screen. Start here for a quick prototype; move to Claude Code when you need control over the code itself.

### Skills as snippet libraries

Save reusable component patterns and API-integration templates as Claude **Skills**, then recall them on command so every build uses the same, tested building blocks instead of starting from scratch.

# SESSION 2 – CLAUDE AS THE BRAIN

What makes an app “agentic”, and the architecture underneath it.

### What makes an app “agentic”

An agentic app doesn’t just call the model once — Claude takes the user’s input, **decides to call a tool**, integrates the result and responds. The tool-calling loop lives inside the product, not in a chat window.

### App architecture with the Claude API

The shape is always the same: **frontend → an API layer → Claude with MCP tools → a database → back to the frontend**. Draw this before you build; every agentic app is a variation of this one diagram.

### Supabase — the backend

A Postgres database with auth, real-time updates and edge functions. It is where your app’s data lives, and Claude can read and write it through an MCP — so the product has **state**, not just answers.

### Connectors inside apps

Wire a user’s own Notion or Drive into your app so Claude has **personalised context per user** — the same answer engine, grounded in each person’s own data.

# SESSION 2 – SHIP & SCOPE

Deploy to a live URL, and plan the capstone you'll ship next week.

### Emergent

Agentic app scaffolding: describe the agent's **behaviour** and get the app architecture and code. A faster start when the product is an agent, not a page.

### Replit

Cloud deployment: it turns localhost into a **live URL**, handles environment variables and custom domains. This is how your build becomes a link you can share and demo.

### Hugging Face Spaces

Host open-source model integrations alongside Claude for specialised tasks — an image, audio, or fine-tuned model your product needs beyond Claude itself.

### Capstone planning

Define the capstone now: **problem statement, user journey, tech stack, Claude architecture and a build plan**. Then write its CLAUDE.md — the context file that keeps Claude on track through next week's sprint.

# THE MODELS TO REMEMBER

## 1 · THE VIBECODING LOOP

Describe → Generate → Test → Iterate → Own ↻

You describe intent; Claude builds; you test and own the result, then go round again. The skill isn't typing code — it's judging it and steering the next loop.

## 2 · THE AGENTIC APP ARCHITECTURE

Frontend → API layer → Claude + MCP tools → Database → Frontend

User input flows to Claude, which calls tools and reads or writes the database, then surfaces the result in the UI. Every agentic app you build is a version of this.

## 3 · THE BUILD LOOP

Requirements → Architecture → Generate → Test → Debug → Deploy

The repeatable path from idea to a live URL. When a build stalls, name the stage you're in — it tells you the next move.

# YOUR TOOLKIT THIS WEEK

## BUILD BRAIN

**Claude** — Claude Code, Projects, Skills, MCPs and the API as your engineering intelligence.

**CLAUDE.md** — the context file that keeps Claude on track across the whole build.

## EDITORS & SCAFFOLDERS

**Cursor** — AI editor: tab-complete, inline edit, composer.

**Lovable · Bolt.new** — prompt-to-app scaffolding.

**Emergent** — agentic app scaffolding from a behaviour description.

## BACKEND & DEPLOY

**Supabase** — Postgres, auth, real-time — the backend.

**Replit** — localhost to a live URL.

**GitHub** — version control and deploy triggers.

**Hugging Face Spaces** — host open-source model integrations.

**Carry it forward**

Next week you ship: the scope and skeleton you build now become your Week 6 Demo Day build plan.

# KEY TERMS

The week's vocabulary, in one line each.

**Vibecoding** — building by describing intent and steering the AI, instead of typing every line yourself.

**Claude Code** — an agentic coding tool that reads your project, edits files and runs commands — with your approval on each change.

**CLAUDE.md** — a context file at your repo root (goal, stack, conventions, never-do) that Claude Code reads every session.

**MCP** — Model Context Protocol — the standard way Claude plugs into tools like GitHub, Supabase and APIs.

**Agentic app** — a product where Claude takes input, decides to call a tool, uses the result and responds — a loop inside the app.

**Scaffold** — the first generated skeleton of an app — folders, pages and a database — that you then build on.

**Supabase** — a hosted Postgres database with auth and real-time — a common backend for AI apps.

**Deploy** — to put your app on a live URL (e.g. via Replit) so others can use it.

**Capstone** — the real product you scope this week and ship on Demo Day in Week 6.